

Merge Sort

1. Basic Idea :-

- Merge Sort is a divide and conquer Algorithm.
- It divides the array into two halves.
- Recursively Sorts both halves.
- Finally merges the two Sorted halves to get the Sorted array.

2. General Steps :-

- ① If array has 0 or 1 element, it is already sorted.
- ② Find the mid and divide the array into two halves.
- ③ Recursively apply merge sort on both halves.
- ④ Merge the two sorted halves.

3. Algorithm :-

```

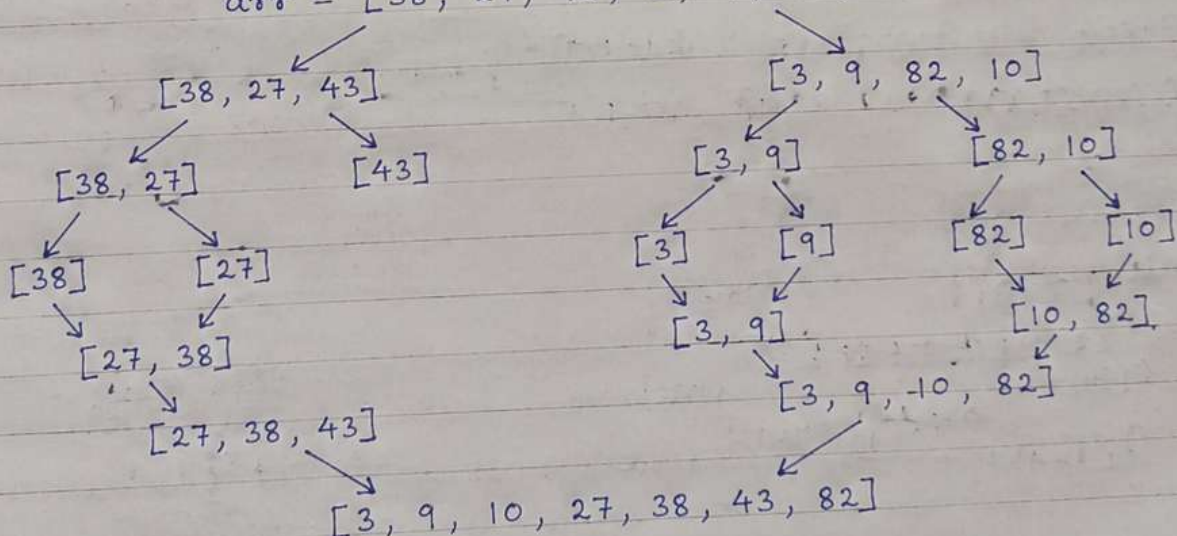
mergeSort(arr, l, r)
{
    if (l >= r) return
    mid = l + (r-l)/2
    mergeSort(arr, l, mid)
    mergeSort(arr, mid+1, r)
    merge(arr, l, mid, r)
}
  
```

Time Complexity

→ Best, Avg, Worst
= $O(n \log n)$

4. Example :-

arr = [38, 27, 43, 3, 9, 82, 10]



```
01_Merge_Sort.cpp 02_Merge_Sort_Medium.cpp 03_Quick_Sort.cpp X 04 ...  
Source_Code > Quick_Sort > 03_Quick_Sort.cpp > main()  
4 int partition(int arr[], int s, int e)  
23 while (i < pivotIndex && j > pivotIndex)  
29 j--;  
30  
31 if (i < pivotIndex && j > pivotIndex)  
32 {  
33 swap(arr[i], arr[j]);  
34 i++;  
35 j--;  
36 }  
37 }  
38 return pivotIndex;  
39  
40 }  
41  
42 void quickSort(int arr[], int s, int e)  
43 {  
44 if (s >= e)  
45 return;  
46  
47 int p = partition(arr, s, e);  
48  
49 quickSort(arr, s, p - 1);  
50 quickSort(arr, p + 1, e);  
51 }  
52  
53 int main()  
54 {  
55 int n;  
56 cin >> n;  
57  
58 int arr[100000];  
59  
60 for (int i = 0; i < n; i++)  
61 cin >> arr[i];  
62  
63 quickSort(arr, 0, n - 1);  
64  
65 for (int i = 0; i < n; i++)  
66 cout << arr[i] << " ";  
67  
68 return 0;  
69 }  
TERMINAL  
PS C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Merge_Sort>  
  
cd "C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Quick_Sort\" ; if ($?) { g++ 03_Quick_Sort.cpp -o 03_Quick_Sort.exe ; if ($?) { .\03_Quick_Sort.exe }  
3  
12  
32  
11  
11 12 32  
PS C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Quick_Sort>
```

```
01_Merge_Sort.cpp 02_Merge_Sort_Medium.cpp 03_Quick_Sort.cpp X 04 ...
Source_Code > Quick_Sort > 03_Quick_Sort.cpp > main()
4 int partition(int arr[], int s, int e)
23 while (i < pivotIndex && j > pivotIndex)
29 j--;
30
31 if (i < pivotIndex && j > pivotIndex)
32 {
33     swap(arr[i], arr[j]);
34     i++;
35     j--;
36 }
37 }
38 return pivotIndex;
39
40 }
41
42 void quickSort(int arr[], int s, int e)
43 {
44     if (s >= e)
45         return;
46
47     int p = partition(arr, s, e);
48
49     quickSort(arr, s, p - 1);
50     quickSort(arr, p + 1, e);
51 }
52
53 int main()
54 {
55     int n;
56     cin >> n;
57
58     int arr[100000];
59
60     for (int i = 0; i < n; i++)
61         cin >> arr[i];
62
63     quickSort(arr, 0, n - 1);
64
65     for (int i = 0; i < n; i++)
66         cout << arr[i] << " ";
67
68     return 0;
69 }
```

TERMINAL

PS C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Merge_Sort>

cd "C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Quick_Sort" ; if (\$?) { g++ 03_Quick_Sort.cpp -o 03_Quick_Sort ; if (\$?) { .\03_Quick_Sort } }

11 12 32

PS C:\Users\cmsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Quick_Sort>

```

Source_Code > Heap_Sort > 05_Heap_Sort.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  void heapify(int arr[], int n, int i)
5  {
6      int largest = i;
7      int left = 2 * i + 1;
8      int right = 2 * i + 2;
9
10     if (left < n && arr[left] > arr[largest])
11         largest = left;
12
13     if (right < n && arr[right] > arr[largest])
14         largest = right;
15
16     if (largest != i)
17     {
18         swap(arr[i], arr[largest]);
19         heapify(arr, n, largest);
20     }
21 }
22
23 void heapSort(int arr[], int n)
24 {
25     for (int i = n / 2 - 1; i >= 0; i--)
26         heapify(arr, n, i);
27
28     for (int i = n - 1; i > 0; i--)
29     {
30         swap(arr[0], arr[i]);
31         heapify(arr, i, 0);
32     }
33 }
34
35 int main()
36 {
37     int n;
38     cin >> n;
39
40     int arr[100000];
41
42     for (int i = 0; i < n; i++)
43         cin >> arr[i];
44
45     heapSort(arr, n);
46
47     for (int i = 0; i < n; i++)
48         cout << arr[i] << " ";

```

```

TERMINAL
Code - Heap_Sort
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module> cd "c:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Heap_Sort\" ; if ($?) { g++ 05_Heap_Sort.cpp -o 05_Heap_Sort } ; if ($?) { .\05_Heap_Sort }
4
12
43
12
45
12 12 43 45
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Heap_Sort>

```


01_Merge_Sort.cpp x 02_Merge_Sort_Medium.cpp 03_Quick_Sort.cpp 04

source_code > Merge_Sort > 01_Merge_Sort.cpp > main()

```
1 #include <iostream>
2 using namespace std;
3
4 void merge(int arr[], int s, int mid, int e)
5 {
6     int len1 = mid - s + 1;
7     int len2 = e - mid;
8
9     int *first = new int[len1];
10    int *second = new int[len2];
11
12    int k = s;
13
14    for (int i = 0; i < len1; i++)
15        first[i] = arr[k++];
16
17    k = mid + 1;
18
19    for (int i = 0; i < len2; i++)
20        second[i] = arr[k++];
21
22    int index1 = 0;
23    int index2 = 0;
24    k = s;
25
26    while (index1 < len1 && index2 < len2)
27    {
28        if (first[index1] < second[index2])
29            arr[k++] = first[index1++];
30        else
31            arr[k++] = second[index2++];
32    }
33
34    while (index1 < len1)
35        arr[k++] = first[index1++];
36
37    while (index2 < len2)
38        arr[k++] = second[index2++];
39
40    delete[] first;
```

TERMINAL

Code - Merge_Sort + -

```
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Heap_Sort> cd "c:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Merge_Sort\" ; if ($?) { g++ 01_Merge_Sort.cpp -o 01_Merge_Sort } ; if ($?) { .\01_Merge_Sort }
4
43
23
54
11
11 23 43 54
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-3-Module\Source_Code\Merge_Sort>
```

05_Heap_Sort.cpp

06_Heap_Sort_Medium.cpp X

D v ⚙ □ ...

TERMINAL

Code - Heap_Sort + v □ □ ... |

Source_Code > Heap_Sort > 06_Heap_Sort_Medium.cpp > main()

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  void heapify(int arr[], int n, int i)
6  {
7      int largest = i;
8      int left = 2 * i + 1;
9      int right = 2 * i + 2;
10
11     if (left < n && arr[left] > arr[largest])
12         largest = left;
13
14     if (right < n && arr[right] > arr[largest])
15         largest = right;
16
17     if (largest != i)
18     {
19         swap(arr[i], arr[largest]);
20         heapify(arr, n, largest);
21     }
22 }
23
24 void heapSort(int arr[], int n)
25 {
26     for (int i = n / 2 - 1; i >= 0; i--)
27         heapify(arr, n, i);
28
29     for (int i = n - 1; i > 0; i--)
30     {
31         swap(arr[0], arr[i]);
32         heapify(arr, i, 0);
33     }
34 }
35
36 int main()
37 {
38     int n;
39     cin >> n;
40
41     int arr[100000];
42
43     double sum = 0;
44
45     for (int i = 0; i < n; i++)
46     {
47         cin >> arr[i];
48         sum += arr[i];
49     }
```

ar-Week-3-Module\Source_Code\Heap_Sort>

```
● d "c:\Users\omsin\OneDrive\Desktop\3rd-Ye
ar-Week-3-Module\Source_Code\Heap_Sort\"
; if ($?) { g++ 06_Heap_Sort_Medium.cpp -
o 06_Heap_Sort_Medium } ; if ($?) { .\06_
Heap_Sort_Medium }
4
12
43
54
11
Sorted: 11 12 43 54
Fastest: 11
Slowest: 54
Average: 30.00
Cases Faster Than Average: 2
Percentage: 50.00%
○ PS C:\Users\omsin\OneDrive\Desktop\3rd-Ye
ar-Week-3-Module\Source_Code\Heap_Sort>
```